CIS 3300 · OVERVIEW

Inheritance and Polymorphism

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Text:** *Core Java, Volume I* (13th ed.), Chapter 5, with a look back at Chapter 6, Section 6.1, for interfaces
- **Materials:** four short slide decks, four parts of class notes, a code tutorial with five examples, and the finished code for each example
- **Prerequisite you already have:** classes, objects, and interfaces (including default methods); static members; final fields, records, and enums; and `ArrayList` and file I/O

1. The short version

So far, our classes have related to each other in two ways: one class uses another, or one class holds another. This unit adds the third relationship, inheritance, in which one class is a more specialized version of another. A `Manager` is an `Employee`, so instead of rewriting everything an employee has, the `Manager` class extends `Employee` and writes only what is different.

Inheritance leads straight to polymorphism, which is the real reason it matters. One array, one `ArrayList`, or one method parameter of the superclass type can hold every kind of employee, and each object still behaves the way its own class says it should. Once objects of many classes share a list, you also need ways to get back to the specific class, which is where casting and `instanceof` come in.

The unit moves from the basics toward the subtle parts: abstract classes, the `final` keyword, the order in which constructors run, and finally multiple inheritance, including why Java allows it for interfaces but not for classes. Most of our class time is spent in code. The slides give the vocabulary, and the examples are where the ideas become real. One of the examples is a problem you will help solve in class.

2. What We Will Cover

- **Inheritance basics.** The three relationships among classes (uses-a, has-a, is-a), superclass and subclass vocabulary, `extends` and `super`, what a subclass does and does not inherit, overriding with `@Override`, overriding compared with overloading, access levels including `protected` and packages, and `Object` as the root of every class.
- **Polymorphism and casting.** The substitution principle, declared type compared with actual type, dynamic and static binding, polymorphic arrays, lists, and method parameters, upcasting and downcasting, `ClassCastException`, `instanceof` with pattern matching, `instanceof` compared with `getClass`, and `equals` in a hierarchy.
- **Abstract classes, final, and constructors.** Abstract classes and methods, final methods

and classes, constructor chaining, the order of object creation, why constructors should not call overridable methods, and serialization with a non-serializable superclass.

- **Solving a problem together.** A vehicle rental agency that reads its fleet from a CSV file, designed and built in class.
- **Multiple inheritance.** The diamond problem; multiple inheritance of state, behavior, and type; interfaces as shared types; conflicting default methods and the rules Java uses to resolve them; and choosing among an abstract class, an interface, and has-a.

3. Objectives

By the end of this unit, you should be able to:

1. Distinguish dependence (uses-a), aggregation (has-a), and inheritance (is-a), and decide whether an is-a relationship fits a given pair of classes.
2. Write a subclass with `extends`, call a superclass constructor with `super(...)`, and call a superclass method with `super.method()`.
3. Explain what a subclass inherits (state, accessible behavior, and type) and what it does not (direct access to private members, constructors, and identity).
4. Override a method correctly using `@Override`, and explain how overriding differs from overloading.
5. Choose among `public`, `protected`, package-private, and `private` access, and predict which code can see a member across packages and subclasses.
6. Explain the difference between a variable's declared type and an object's actual type, and use it to predict which method runs.
7. Use a superclass or interface type for arrays, `ArrayList`s, parameters, and return values so that one piece of code works for many classes.
8. Downcast safely with `instanceof`, including pattern matching, and explain when a cast fails at compile time and when it fails at run time.
9. Compare `instanceof` with `getClass`, and write an `equals` method that behaves correctly in a hierarchy.
10. Design an abstract class with abstract methods, and use `final` on methods and classes to protect a design.
11. Trace the order in which static blocks, field initializers, and constructors run when a subclass object is created, and explain why a constructor should not call an overridable method.
12. Design and implement a small class hierarchy from a written problem description.
13. Explain the diamond problem, describe how much multiple inheritance Java allows for state, behavior, and type, and resolve a conflict between two default methods.

4. How to Show Up Ready

- **Read ahead.** Skim Core Java, Volume I, Chapter 5, Section 5.1, before the first class, and keep reading through Section 5.2 as the unit goes on.
- **Review the earlier unit.** Look again at your `toString`, `equals`, and `hashCode` methods and the `Serviceable` interface. This unit explains what you were really doing when you overrode them.
- **Bring a working Eclipse setup.** We will create packages, run several programs, and read compiler errors together. Reading the error message carefully is part of the lesson.
- **Work through the code tutorial.** Type the examples yourself instead of only reading them. The finished code for each example is there if you get stuck or miss a class.
- **Come ready to design.** For the problem-solving example, think about what classes you would create before you write any code.